Laboratory Guide

Lesson 04 — Discovery and Interrogation of BLE Devices

Communication Technologies for the Internet of Things

Academic year 2026–2027

Catarina Silva

ESTGA, University of Aveiro

Estimated duration: 2 hours

Contents

Context **2**

Execution Modes: With or Without Hardware **2**

Learning Outcomes **3**

1 Adapter Preparation **4**

2 Device Discovery **5**

3 Interrogating the GATT Hierarchy **6**

4 Advertising and Connection **7**

5 Analysing an HCI Capture (optional, 20 min) **9**

Questions for Discussion **9**

References **9**

Context

The lecture described the link layer, the advertising and scanning states, and the organisation of data into services and characteristics under GATT. This session observes those mechanisms in operation, on real devices, using the BlueZ stack tools presented in the slides.

No software is developed. The commands used are provided and executed in a console, and the student's work consists of interpreting what is returned and relating it to the concepts from the lecture.

Material

- Raspberry Pi with BlueZ installed, acting as the central device.
- One smartphone per group, with the nRF Connect application installed, acting as the BLE peripheral. No additional equipment is required.
- Without this equipment, follow the simulated mode described in "Execution Modes".

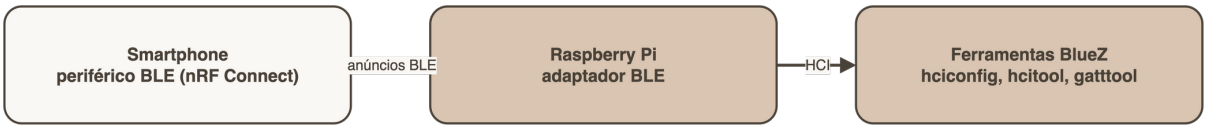


Figure 1: *Architecture of the exercise*

Execution Modes: With or Without Hardware

The guide can be done in three modes. The commands and the interpretation of results are the same; only the source of the devices changes.

Mode	Equipment	How it works
A. Physical	Raspberry Pi + smartphone with nRF Connect	Real commands; "phone" is the smartphone
B. Mixed	Only the smartphone, or only the Pi	Part real, part simulated (e.g. the Pi scans but the peripheral is simulated)
C. Simulated	Any computer with Python 3	`ble_sim.py` mimics the tools; "phone" is the `phone adv ...` command

Setting up the simulated mode

Download the `codigo.zip` file (the *Source Code* button of the lecture on the course page), extract it and, in the extracted folder, run:

```
python3 ble_sim.py          # on Windows: python ble_sim.py
```

There are no dependencies: Python 3.8 or later on Linux, macOS or Windows is enough. The simulator opens a console that accepts `hciconfig`, `sudo hcitool lescan` and `gatttool -b <address> [-t random] -I`, with the same options and an output format very close to the real one. The simulated adapter starts *down*, as on a freshly powered Pi.

Simulated devices

- `TCIoT-Sensor` (public address): connectable; Generic Access, Battery (with notifications) and a vendor service with temperature in hundredths of a degree.

- `TCIoT-Beacon`: **non**-connectable advertiser, data in the advertisement itself.
- `Band-Mock`: tracker with a **random** address that changes at every scan; the name arrives only in the scan response.
- `Telemovel-Aluno`: stands in for nRF Connect. Controlled with `phone adv on`, `phone adv off`, `phone adv conn` and `phone adv nonconn`.

Installing the analysis tools (modes B/C and section 5)

- **Wireshark** (Windows, macOS and Linux): `https://www.wireshark.org/download.html`; on Debian/Ubuntu, `sudo apt install wireshark`.
- **Bumble** (optional, Python Bluetooth stack with virtual controllers): `python3 -m venv venv && venv/bin/pip install bumble`. For example, `venv/bin/bumble-show -format snoop captura_ble.btsnoop`.

Advanced alternatives (not required)

A recent Android emulator includes a virtual Bluetooth controller; on Linux, the `hci_vhci` module and BlueZ's test emulator create virtual adapters that `bluetoothctl` recognises. They depend on the host system and were not validated for this session; use them only as an exploration project.

Learning Outcomes

- Identify BLE devices in the vicinity and distinguish those advertising in a connectable manner from those merely broadcasting information;
- Traverse the GATT hierarchy of a real device, from service down to characteristic value;
- Relate the observed identifiers (UUID and *handle*) to the concepts of attribute type and address presented in the lecture;
- Recognise, in practice, the difference between advertising-based and connection-based communication.

1. Adapter Preparation

15 min

Objective

Confirm the presence of the BLE adapter and bring it to an operational state.

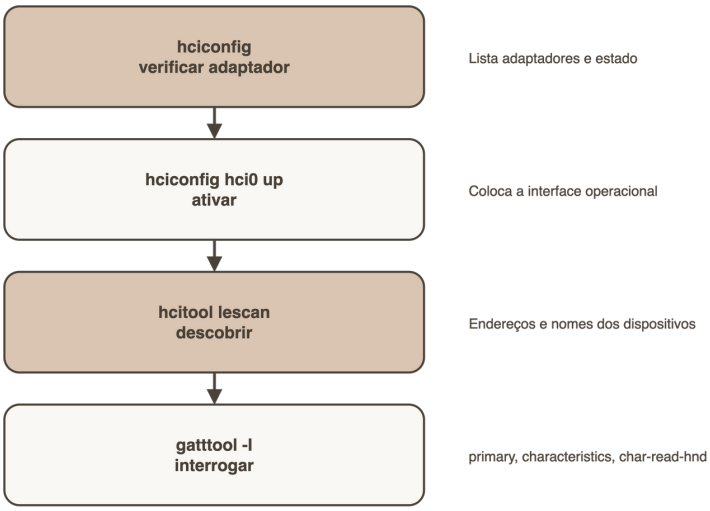


Figure 2: Tool usage sequence

```
hciconfig
sudo hciconfig hci0 up
hciconfig -a hci0
```

1. Run `hciconfig` with no arguments and record the name of the interface shown, usually `hci0`, together with its address.
2. Enable the interface and query the state again with the `-a` option.
3. Record the HCI version reported and compare it with the corresponding Bluetooth specification version.

Question

Is the address shown by `hciconfig` public or random? What consequence does this distinction have for the possibility of tracking the device over time?

Hint

Adapter addresses are generally public. The addresses of a modern smartphone almost never are: see section 2.

BlueZ configuration (optional, Raspberry Pi)

If the `hci0` interface does not appear or the Pi is shared by several groups:

```
sudo nano /etc/hostname      # nome distinto por grupo
sudo apt install bluez bluez-hcidump
sudo nano /lib/systemd/system/bluetooth.service
#  ExecStart=/usr/lib/bluetooth/bluetoothd --experimental
```

```
sudo nano /etc/bluetooth/main.conf      # ControllerMode = le
sudo systemctl daemon-reload
sudo systemctl restart bluetooth
```

The `le` mode leaves the adapter operating in BLE only, which simplifies scanning. In simulation, skip this step.

2. Device Discovery

30 min

Objective

Observe advertising traffic and characterise the devices present in the room.

```
sudo hcitool lescan
```

1. Run the scan for about 30 seconds and interrupt with `Ctrl+C`.
2. Record in a table the addresses and names obtained. Note how many devices present a name and how many appear with an address only.
3. On the smartphone, open nRF Connect and enable peripheral mode (*Advertiser*), with a name identifiable by the group. Repeat the scan and confirm that the device now appears in the list.
4. Disable advertising on the smartphone and repeat the scan a third time.

Address	Name	Observations

Random addresses

Smartphone addresses are normally **resolvable private** addresses: they change over time. So when the scan is repeated the phone may appear with another address; use the most recent one in the following steps. A tracker that kept its address would be trackable; one that changes it no longer is (see the lecture).

Also record: a device that appears as `(unknown)` and, right after, with a name corresponds to an advertisement without a name followed by the active-scan response.

Relation to the lecture

The scan performed by `hcitool lescan` is active: besides listening on channels 37, 38 and 39, it issues scan requests to which the advertiser replies with additional information. That is why some devices present a name and others do not.

Alternative with bluetoothctl

`hcitool` is deprecated. With the current BlueZ tool:

```
bluetoothctl
[bluetooth]# scan on
[bluetooth]# scan off
```

Besides address and name, it shows RSSI, transmit power and the advertisement data, which allows estimating the approximate distance with the model from the lecture. In simulated mode, use `hcitool lescan`.

3. Interrogating the GATT Hierarchy

45 min

Objective

Traverse the services and characteristics of a real peripheral and read the value of a characteristic.

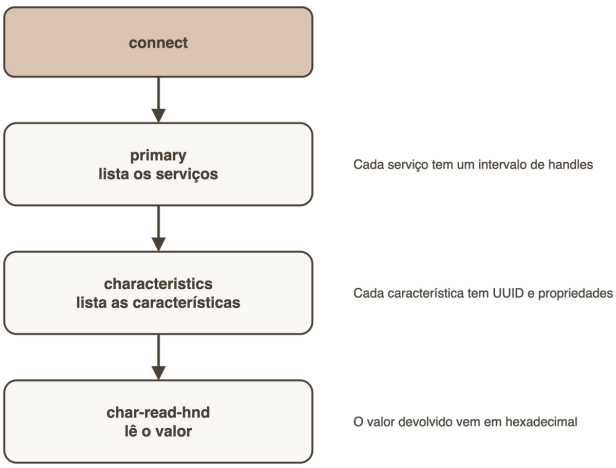


Figure 3: GATT exploration sequence

Keep the smartphone advertising and, from the Raspberry Pi, run:

```
sudo gatttool -I -b <DEVICE_ADDRESS>
```

In interactive mode, issue in sequence:

```
connect
primary
characteristics
char-read-hnd <handle>
disconnect
```

1. Record the services returned by `primary`, noting for each the UUID and the range of *handles*.
2. Record the characteristics returned by `characteristics`, noting the value *handle*, the properties and the UUID.
3. Using `char-read-hnd`, read the value of a characteristic whose properties include reading. The result is shown in hexadecimal.

4. Identify, among the UUIDs observed, those corresponding to services standardised by the Bluetooth SIG and those specific to the manufacturer.

Random addresses and the -t random option

If the address is random (the case of smartphones and `Band-Mock`), `gatttool` must be told: `sudo gatttool -b <address> -t random -I`. Without this option the connection fails with *Connection refused*, and the failure does not indicate a problem with the device.

5. List all attributes with `char-desc`. Using the UUIDs `0x2800`, `0x2803` and `0x2902`, identify service declarations, characteristic declarations and notification configuration descriptors. Compare the layout with the GATT hierarchy figure.
6. Read the battery level characteristic (UUID `0x2A19`) and convert the result from hexadecimal to decimal. For example, `57` corresponds to 87 %.
7. Enable notifications: write `0100` to the battery `0x2902` descriptor, with `char-write-req <descriptor handle> 0100`. Observe the `Notification handle = ...` lines that arrive without any new request, and disable them by writing `0000`.
8. Read the vendor characteristic holding the temperature (two bytes, *little-endian*, in hundredths of a degree) and compute the value in °C.

Question

What is the advantage of notifications over repeating `char-read-hnd`? When is an indication preferable?

Reading the identifiers

The 16-bit UUIDs, shown in the form `0x180A`, correspond to services adopted by the Bluetooth SIG. The 128-bit UUIDs identify services defined by the manufacturer. This distinction makes it immediately apparent which part of the device follows standardised profiles.

Further work

With the nRF Connect application acting as client, connect to a second device and compare the information presented by the application with that returned by `gatttool`. The application automatically resolves standardised UUIDs into readable names; identify which.

4. Advertising and Connection

20 min

Objective

Distinguish experimentally the two communication paradigms studied.

1. Configure nRF Connect to broadcast data in the advertising packet itself, without allowing connection.
2. Confirm by scanning that the device remains visible and that the data is readable with no connection established.
3. Attempt to connect with `gatttool` and record the result.
4. Restore connectable mode and confirm that connection becomes possible.

Simulated mode

Instead of nRF Connect, use `phone adv nonconn`, repeat `sudo hcitool lescan` to confirm the phone is still visible and try `gatttool -b <address> -t random -I` followed by `connect`. Restore with `phone adv conn`. The address changes when advertising is reconfigured: read the new value from the scan.

5. Analysing an HCI Capture (optional, 20 min)

Alternative or complement to live scanning

Objective

Recognise advertisements, connection and GATT transactions in a capture file, such as the one Android produces with the *HCI snoop log*.

The file `captura_ble.btsnoop` (included in `codigo.zip`) holds 26 packets from a session equivalent to sections 2 and 3. Open it in Wireshark (*File* → *Open*) and use the filters `bthci_evt` (HCI events) and `btatt` (attribute protocol).

1. How many distinct advertisers appear? Which are connectable and which is not? The event type of the advertising report says so (`ADV_IND`, `ADV_NONCONN_IND`, `SCAN_RSP`).
2. Which address has a random type, and why does its name come in a separate packet?
3. Identify the connection request, the agreed connection interval (in 1.25 ms units) and the connection *handle*.
4. Filter by `btatt.opcode == 0x1b` and record the *handle* and value of each notification. Which earlier write enabled them?
5. For the battery read request, convert the returned value to a percentage.

Alternatively, without Wireshark: `bumble-show -format snoop captura_ble.btsnoop`. To produce a real capture, enable the *Bluetooth HCI snoop log* on Android (developer options) or run `sudo btmon -w captura.btsnoop` on the Raspberry Pi during the earlier exercises.

Questions for Discussion

1. What information was obtainable about third-party devices without any authorisation, and what are the implications of that fact?
2. Why is advertising-based broadcasting more energy-efficient than connection-based communication?
3. Is the activity tracker case presented in the lecture observable in the room? How could it be verified?
4. Which pairing method would be used between the Raspberry Pi and a sensor with neither display nor keyboard, and what risk does that entail?
5. Compare `hcitool`/`gatttool` with `bluetoothctl`: why did BlueZ mark them as deprecated, and what are the risks of depending on them?
6. A temperature sensor sends one reading per minute. Justify whether it should use advertising-based communication or connection with notifications, considering energy, reliability and privacy.

References

- Gonçalves, P. *Apresent4 — Bluetooth*, TCIoT, previous editions.
- Bluetooth SIG. *Bluetooth Core Specification*.
- `hciconfig` documentation: `https://helpmanual.io/man1/hciconfig/`
- `gatttool` documentation: `https://helpmanual.io/man1/gatttool/`

- BlueZ. *bluetoothctl* and *btmon*: `https://www.bluez.org/`
- Wireshark, *Bluetooth*: `https://wiki.wireshark.org/Bluetooth`
- Bumble, Python Bluetooth stack: `https://google.github.io/bumble/`